

Memory Management & Linux Commands

1 Memory Management

1.1 Physical and Logical Address

- Physical address: actual location in main memory (RAM); the address memory hardware understands. Set of all physical addresses used by a program = physical address space.
- Logical (virtual) address: address generated by the CPU during execution; used by the program to access its own instructions/data. Set of all logical addresses used = logical address space.
- MMU (Memory Management Unit) translates logical → physical address at runtime.

Important Questions

1. **Q:** Differentiate between logical and physical address.
A: Logical address is generated by the CPU during program execution and used by the program itself; physical address is the actual RAM location the hardware understands. Logical address is translated to physical address by the MMU.

1.2 Address Binding

- Address binding = mapping from addresses associated with a program to actual memory addresses.
- **Compile time:** memory location known at compile time ⇒ absolute code generated; logical address = physical address.
- **Load time:** location not known at compile time ⇒ relocatable code generated at compile time, converted to absolute code at load time.
- **Run time:** process may move between memory segments during execution ⇒ relocatable code at compile time, converted to absolute code at run time. Performed by the MMU (relocation/base register).

Important Questions

1. **Q:** What is address binding? List its types with the key distinguishing condition for each.
A: Address binding is the mapping of a program's addresses to memory addresses. Compile time: locations known at compile time, generates absolute code. Load time: locations unknown at compile time, resolved at load time. Run time: process can move between memory segments during execution; resolved by MMU using a relocation register.
2. **Q:** Which type of address binding is performed by the MMU?
A: Run-time address binding, using a relocation (base) register.

1.3 Bare Machine

- Computer with no operating system; programs fed directly in machine language.
- Entire memory given to the user process, which runs in kernel mode with full control.
- No protection mechanism needed (no OS/other process resident) ⇒ no overhead.
- Logical address = physical address ⇒ no address translation, no memory management strategy required.
- Drawback: user must supply all services normally provided by an OS.

Important Questions

1. **Q:** Why is no memory management strategy required in a bare machine?

A: The whole memory belongs to a single user process running in kernel mode; logical address equals physical address, so no translation or protection is needed.

1.4 Contiguous Memory Management

Single Partition

- Memory split into two parts: one for the OS (resident monitor), one for the user process.
- OS placed at the end of memory where the interrupt vector is located.
- Hardware support: bounds register + two CPU modes (user/supervisor). User-mode memory reference is checked against the bounds register; violation $\Rightarrow$ interrupt to OS.
- Only one process executes at a time; process must fit in available memory or it errors out.
- Advantage: easy to implement, minimal hardware needed.
- Disadvantage: memory not fully utilized (unused allocated space wasted); process size capped by memory size.

Multiple Partitions

- Allows several processes in memory simultaneously; memory divided into fixed or variable partitions.
- **Fixed-sized partitions:** any process $\leq$ partition size can load into any free partition. OS tracks which partition holds which process. Easy to implement (all partitions equal size).
 - Drawback: degree of multiprogramming limited by number of partitions; inefficient for small processes; causes **internal fragmentation**. (Obsolete technique.)
- **Variable-sized partitions:** whole memory starts as one large hole; a request is satisfied from a hole big enough to hold it; remaining memory stays available. Free-storage list (linked list of holes: size + address of next hole) tracks free space.
 - Problem: scattered free holes may together be big enough for a request but can't be used $\Rightarrow$ **external fragmentation**.
 - Fix for internal fragmentation: allocate smallest hole large enough for the request (reduces waste but doesn't eliminate it).
 - Fix for small-job starvation (large partition's queue empty, small partition's queue full): use a single input queue instead of one queue per partition.

Fragmentation

- **Internal fragmentation:** wasted memory inside an allocated block; occurs in fixed-size allocation (fixed partitioning, paging); cannot be reduced by compaction.
- **External fragmentation** (checker-boarding): enough total free memory exists but scattered in non-contiguous blocks; occurs in variable-size allocation (dynamic partitioning, segmentation); reduced/eliminated by **compaction** (shuffling memory to merge holes into one large hole). Compaction is expensive in CPU time $\Rightarrow$ rarely used.

Partition Selection Algorithms

- **First fit:** allocate the first hole large enough. Fastest (least searching).
- **Best fit:** allocate the smallest hole $\geq$ process size. Slower (searches whole list); leaves smallest unusable leftover holes.

- **Worst fit:** allocate the largest hole. Leaves largest leftover holes; simulation shows it is not effective at reducing wastage.
- **Next fit:** like first fit, but search resumes from where the previous allocation ended (not from the start).

Important Questions

1. **Q:** Distinguish between internal and external fragmentation.
A: Internal fragmentation is wasted space inside an allocated block (fixed-size allocation, e.g. fixed partitioning/paging); the unused memory can't be given to another process, and compaction doesn't fix it. External fragmentation is wasted space between allocated blocks (variable-size allocation, e.g. dynamic partitioning/segmentation); enough free memory exists in total but it's non-contiguous, and compaction can reduce/eliminate it.
2. **Q:** Explain first fit, best fit, worst fit and next fit partition selection algorithms.
A: First fit allocates the first large-enough hole (fastest). Best fit allocates the smallest hole $\geq$ request size (slower, more small unusable leftovers). Worst fit allocates the largest hole (leaves large leftovers, not efficient). Next fit is first fit but resumes searching from the last allocation point instead of the start.
3. **Q:** What is compaction and why is it rarely used?
A: Compaction relocates memory contents so all free holes are merged into one large contiguous block, solving external fragmentation. It is rarely used because it consumes significant CPU time.
4. **Q:** Given memory partitions of 100K, 500K, 200K, 300K, and 600K (OS occupies 200K in the first partition), place processes of 212K, 417K, 112K and 426K in order using first fit, best fit, and worst fit. Which is most efficient?
A: Work through each algorithm against the free-hole list after removing the OS's 200K: first fit places each process in the first hole $\geq$ its size scanning from the top, best fit picks the smallest sufficient hole each time, worst fit always picks the largest hole. Best fit and first fit generally place all four processes with smaller leftover waste than worst fit, which quickly exhausts the largest holes; on this input, best fit and first fit succeed in placing all processes while worst fit fails to place the last (426K) process, making first fit/best fit more efficient here.
5. **Q:** Why is the fixed-partitioning technique no longer in use?
A: It limits the degree of multiprogramming to the number of partitions, wastes memory for small processes, and causes internal fragmentation.

1.5 Non-Contiguous Allocation: Paging

- Physical memory divided into fixed-size **page frames**; logical memory divided into same-size **pages**. A process's pages can load into any free frames, not necessarily contiguous.
- Logical address = page number (p , high-order bits) + page offset (d , low-order bits). Page size is a power of 2 so this split is easy.
- **Page table:** maps page number $\rightarrow$ frame number, one per process. Page number indexes the table.
- Translation: CPU generates logical address $\rightarrow$ MMU looks up frame number in page table using page number $\rightarrow$ frame number + offset = physical address. Page and frame offsets are identical (no mapping needed for offset).
- Advantage: no external fragmentation (allocation always in fixed frame units).
- Disadvantage: internal fragmentation possible (last frame of a process may not be fully used if process size isn't a multiple of page size).

Page Allocation Tracking

- **Memory-Map Table (MMT)**: static table, one entry per frame, marks free/allocated. $f = m/p$ (frames = memory size / page size). Simple but search time increases as usage increases; deallocation updates multiple entries.
- **Free list (linked list)**: free frames as a linked list; allocate by removing first n nodes; on process termination, released frames added to the front. Allocation/deallocation time independent of memory utilization; higher memory overhead than MMT but faster.

Hardware Support for Page Tables

- **In registers**: very fast, no memory reference needed; not suitable for large tables (registers expensive); all registers reloaded on every context switch (slow).
- **In main memory**: Page Table Base Register (PTBR) holds start address of the table; only PTBR reloaded on context switch (fast switch), but every memory access needs two memory references (page table, then actual memory) $\Rightarrow$ slower access.
- **TLB (Translation Look-aside Buffer)**: small associative memory inside the MMU holding recent page table entries. **TLB hit**: frame number found directly. **TLB miss**: page table consulted in memory, entry added to TLB (may replace an existing entry if full). PID added to each TLB entry to disambiguate entries from different processes.

Important Questions

1. **Q**: Explain the process of address translation in paging.
A: The logical address is split into page number p and offset d . The MMU uses p to index the page table and retrieve the frame number. This frame number replaces p , and combined with offset d (unchanged) forms the physical address.
2. **Q**: A system generates a 16-bit logical address with page size 4KB. Find the number of bits for the page number and page offset.
A: Page size = 4KB = 2^{12} bytes $\Rightarrow$ offset = 12 bits. Logical address = 16 bits $\Rightarrow$ page number = $16 - 12 = 4$ bits.
3. **Q**: A paged memory system has eight pages of 8KB each and 16 page frames. Compute the physical address for logical address 18325 using the page table (page 2 $\rightarrow$ frame 1011).
A: 8 pages = $2^3 \Rightarrow$ page number = 3 bits; page size 8KB = $2^{13} \Rightarrow$ offset = 13 bits; logical address = 16 bits. 16 frames = $2^4 \Rightarrow$ physical address = 17 bits. $18325 = 0100011110010101_2$: page number = 010 = 2, offset = 0011110010101. Page 2 maps to frame 1011, so physical address = $10110011110010101_2 = 92053$.
4. **Q**: A logical address space has eight pages of 1024 words each, mapped onto 32 physical frames. Find the number of bits in the logical and physical address.
A: 8 pages = $2^3 \Rightarrow$ 3 bits for page number; 1024 words = $2^{10} \Rightarrow$ 10 bits for offset $\Rightarrow$ logical address = 13 bits. 32 frames = $2^5 \Rightarrow$ 5 bits for frame number $\Rightarrow$ physical address = 15 bits.
5. **Q**: Compare storing the page table in registers vs. main memory vs. using a TLB.
A: Registers: fastest, no extra memory access, but not viable for large tables and slow context switches (all registers reloaded). Main memory + PTBR: fast context switch (only PTBR reloaded) but every access needs two memory references, halving effective speed. TLB: caches recent page table entries in the MMU; a TLB hit avoids the extra memory access, a TLB miss falls back to the page table in memory and updates the TLB; PIDs are added to entries to distinguish processes.
6. **Q**: Why does paging not cause external fragmentation but can cause internal fragmentation?
A: Memory is always allocated in fixed-size frames, so any free frame can serve any request — no

scattered unusable holes (no external fragmentation). But if a process's size isn't an exact multiple of the page size, its last frame is only partially used, wasting space within that frame (internal fragmentation).

1.6 Virtual Memory and Demand Paging

- **Virtual memory:** lets processes execute even if larger than physical RAM, by using secondary storage as an extension of RAM; only required pages are loaded, rest stay on disk.
- Needed because RAM is limited and modern applications often need more memory than is physically available; without it, oversized programs can't run and fewer programs run concurrently.
- Implemented via demand paging or demand segmentation; demand paging is more common (easier to implement).

Demand Paging

- A page is loaded into memory only when needed; pages never accessed are never loaded.
- Requires secondary storage (swap space: contiguous area per process on disk) and a page table with a **valid bit** per entry (1 = page in memory, 0 = only on disk).
- Loading a page from swap space into memory is called **lazy swapping** / **page-in** (not "swap-in", since the whole process isn't loaded).
- **Page fault:** MMU finds valid bit = 0 $\Rightarrow$ raises interrupt $\Rightarrow$ page fault routine checks validity via the process's PCB; if invalid, terminates process with error; if valid: locate a free frame, page-in the desired page, update process/page table, restart the faulting instruction.
- **Pure demand paging:** process starts execution with zero pages in memory; first instruction immediately causes a page fault. Principle: "never bring a page until it is required."
- **Locality of reference:** programs should access nearby pages together to avoid excessive page faults per instruction.
- Advantages: reduces swap time (only needed pages moved), increases degree of multiprogramming, minimizes initial disk overhead, needs no extra hardware.
- **Dirty (modified, M) bit:** set when a page is written to; on replacement, page written back to disk only if dirty bit is set (saves unnecessary writes).

Page Replacement Algorithms

- Evaluated using a **reference string** (ordered list of page references) and counting page faults.
- **FIFO:** replaces the page loaded earliest (oldest swap-in time), via a FIFO queue. Simple but not efficient — ignores usage pattern.
- **Belady's anomaly:** increasing the number of page frames can *increase* page faults under FIFO.
- **Optimal (OPT):** replaces the page that will not be used for the longest time in the future. Fewest possible page faults, but needs future knowledge — not practically usable; used only as a benchmark.
- **LRU:** replaces the page not used for the longest time in the past (approximates OPT using past behaviour). Implemented via a linked list (MRU at head, LRU at tail — overhead: list updates on every reference) or a hardware counter (lowest counter value = least recently used). Fewer faults than FIFO; does not suffer Belady's anomaly.
- OPT and LRU belong to **stack algorithms**, which never exhibit Belady's anomaly.
- **MRU:** replaces the most recently used page. Can suffer Belady's anomaly.

Important Questions

1. **Q:** What is a page fault and how is it handled?

A: A page fault is an interrupt raised by the MMU when the valid bit of the requested page's table entry is 0 (page not in memory). The page-fault routine checks the page's validity from the process's PCB; if invalid it terminates the process with an error, otherwise it finds a free frame, pages in the required page from swap space, updates the process/page table, and restarts the interrupted instruction.

2. **Q:** Explain FIFO page replacement and Belady's anomaly.

A: FIFO replaces the page that has been in memory the longest (earliest swap-in time), tracked via a queue with the oldest page at the front. Belady's anomaly is the counter-intuitive situation where increasing the number of available frames increases the number of page faults under FIFO — e.g. a reference string that causes 9 faults with 3 frames can cause 10 faults with 4 frames.

3. **Q:** Why is the Optimal page replacement algorithm not used in practice despite giving the fewest page faults?

A: It requires knowing which page will be referenced furthest in the future, which isn't available in a running system; it's used only as a theoretical benchmark to judge other algorithms.

4. **Q:** How does LRU approximate the Optimal algorithm, and what are its implementation methods?

A: LRU replaces the page unused for the longest time in the past, assuming past behaviour predicts near-future behaviour. Implemented either via a linked list (MRU at head, LRU at tail; updated on every reference — overhead) or a hardware counter incremented per instruction, with the lowest counter value in the page table identifying the LRU page.

5. **Q:** For reference string 7,0,1,2,0,3,0,4,2,3,0,3,2,3 with 4 page frames, find the number of page faults using MRU replacement.

A: 11 page faults: 4 compulsory faults loading 7,0,1,2; then a fault each time a new page — 3,0,4,3,0,3,2,3 — replaces the most recently used resident page, except on the two occasions (0 and 2) where the requested page is already resident.

6. **Q:** What is the dirty (modified) bit used for in page replacement?

A: It marks whether a page has been written to since being loaded. On replacement, a page is written back to disk only if its dirty bit is set; if clear, it's discarded without a costly disk write.

1.7 Thrashing

- **Definition:** a condition where the OS spends more time swapping pages between RAM and disk than executing processes, due to a very high page fault rate.
- **Causes:** insufficient RAM for the active working sets of running processes; too high a degree of multiprogramming (too many processes competing for memory); poor page replacement decisions (evicting frequently-used pages); large working sets that can't fit in RAM.
- **Detection indicators:** consistently high page fault rate; low CPU utilization with high disk activity; continuous/frequent page replacement; monitoring Page Fault Frequency (PFF) per process against a threshold.
- **Prevention:** increase physical RAM (most effective); reduce the degree of multiprogramming by suspending/swapping out some processes to free memory for the rest.

Important Questions

1. **Q:** Define thrashing and list its causes.

A: Thrashing is excessive page faulting causing the OS to spend more time transferring pages between RAM and disk than executing processes. Causes: insufficient RAM for active working sets, high degree of multiprogramming, poor replacement decisions, and processes with large working sets.

2. **Q:** How does the operating system detect thrashing, since the CPU cannot detect it directly?
A: The OS monitors: a consistently high page fault rate, low CPU utilization combined with high disk activity, frequent/continuous page replacement, and per-process page fault frequency (PFF) against a threshold.
3. **Q:** What techniques prevent thrashing?
A: Increasing physical RAM (most effective, lets more pages stay resident) and reducing the degree of multiprogramming by temporarily suspending or swapping out some processes to free memory for the rest.

1.8 Segmentation

- Implements the user's view of a program as a collection of variable-size segments (main program, routines, variables, etc.), each identified by a segment number/name.
- Logical address = segment number (s) + offset (d), specified by the user/compiler — unlike paging, this split is not done by hardware.
- **Segment table:** maintained by the OS; each entry has **segment base** (starting physical address) and **segment limit** (length). Segment number indexes the table.
- Translation: MMU indexes segment table with s ; if offset $d >$ segment limit $\Rightarrow$ invalid-address error; else physical address = segment base + d .
- Advantages: each segment can have different protection (e.g. procedure = execute-only, array = read-only); allows sharing of code/data (e.g. shared library) between processes via a shared segment.

Important Questions

1. **Q:** How does segmentation differ from paging in how the logical address is divided?
A: In segmentation, the split into segment number and offset is specified by the user/compiler; in paging, the split into page number and offset is performed by the hardware based on address bits.
2. **Q:** Using a segment table where segment 2 has base 2200, limit 780, and segment 4 has base 1650, limit 400, compute the physical address for (a) segment 2, offset 247 and (b) segment 4, offset 439.
A: (a) Offset 247 < limit 780, valid: physical address = 247 + 2200 = 2447. (b) Offset 439 > limit 400: invalid-address error is generated.
3. **Q:** Give two advantages of segmentation over a flat addressing scheme.
A: Different segments can have different protection levels appropriate to their content (e.g. execute-only code, read-only data); segmentation allows sharing of code or data (e.g. a shared library) between processes by placing it in one shared segment.

2 Linux Commands

2.1 df — Disk Free Space

- Reports used/available disk space per filesystem: filesystem name, total size, used, available, mount point.
- Syntax: `df [OPTIONS] [FILE/FILESYSTEM]`.
- Key flags: `-h` human-readable; `-k` kilobytes (default); `-m` megabytes; `-i` inode usage instead of space; `-T` show filesystem type; `-a` include pseudo-filesystems; `--total` show a total line.

2.2 du — Disk Usage

- Recursively summarizes disk usage of files/directories; used to find large files/directories.
- Syntax: `du [OPTIONS] [DIRECTORY]`.

- Key flags: `-h` human-readable; `-s` total only per argument; `-a` show files too, not just dirs; `-c` grand total; `-d N` limit depth to N; `-S` exclude subdirectory sizes.
- Common pattern: `du -sh /path/* | sort -hr` to list largest first.

2.3 tar — Archive Files

- Packages multiple files into one archive; can create/extract/list/modify; often combined with compression; preserves permissions and directory structure.
- Syntax: `tar [OPTIONS] [ARCHIVE] [FILES/DIRECTORIES]`.
- Core flags: `-c` create; `-x` extract; `-t` list; `-f FILE` archive filename; `-v` verbose; `-z` gzip (`.tar.gz`); `-j` bzip2 (`.tar.bz2`); `-J` xz (`.tar.xz`).
- Examples: `create = tar -czvf backup.tar.gz dir/`; `list = tar -tzf backup.tar.gz`; `extract = tar -xzf backup.tar.gz`.

2.4 zip — ZIP Archives

- Creates/manages ZIP-format archives; cross-platform (Windows-compatible); supports password protection; can update existing archives.
- Syntax: `zip [OPTIONS] ARCHIVE.zip [FILES/DIRECTORIES]`.
- Key flags: `-r` recursive; `-e` encrypt with password; `-q` quiet; `-l LF→CRLF`; `-u` update archive; `-d` delete files from archive; `-0` to `-9` compression level (9 = max); `-x` exclude pattern.

2.5 uname — System Information

- Prints OS/kernel information: OS name, kernel version, hardware info.
- Syntax: `uname [OPTIONS]`.
- Key flags: `-a` all info; `-s` kernel name; `-n` hostname; `-r` kernel release; `-v` kernel version details; `-m` hardware/machine name; `-p` processor type; `-i` hardware platform; `-o` OS name.

2.6 chmod — Change File Permissions

- Changes file/directory access permissions; supports numeric (octal) and symbolic notation.
- Syntax: `chmod [OPTIONS] MODE FILE`.
- Octal: $r = 4$, $w = 2$, $x = 1$; digit order = owner, group, others. E.g. 755 = owner rwx, group/others r-x; 644 = owner rw-, others r-; 700 = owner rwx only; 777 = everyone rwx; 666 = everyone rw-.
- Symbolic: u(user)/g(group)/o(others)/a(all) with +/=/ and r/w/x. E.g. `chmod u+x file` adds execute for the owner.

Important Questions

1. **Q:** What is the difference between `df` and `du`?
A: `df` reports filesystem-level free/used disk space (per mounted filesystem); `du` reports disk usage of specific files/directories, recursively, useful for finding what's consuming space within a filesystem.
2. **Q:** Write the command to create a gzip-compressed tar archive of `/home/user/Documents/` and one to extract it.
A: Create: `tar -czvf backup.tar.gz /home/user/Documents/`. Extract: `tar -xzf backup.tar.gz`.
3. **Q:** What does `chmod 750 file` set the permissions to?
A: Owner: read, write, execute (7 = 4+2+1); Group: read, execute (5 = 4+1); Others: no permissions (0).
4. **Q:** How would you create a password-protected ZIP archive of a directory, recursively?
A: `zip -e -r secure.zip /path/to/dir/` — prompts for a password and encrypts the archive.
5. **Q:** Which `uname` flag shows all system information at once, and which shows only the kernel release

version?

A: -a shows all information; -r shows only the kernel release version.